CARBON API

Gestão de Emissão de Carbono

Testes Automatizados com BDD (Gherkin)

Testes de API • Contrato com JSON Schema • Execução em CI

Equipe

Integrante	RM
Gabriel Alves Oliveira	RM562365
Marcos Antão Fernandes Junior	RM563156
Marcos Eduardo Alves de Oliveira Freitas	RM555914
Danilo Ferreira Chaves	RM565765
Hevilyn Romano Garcia	RM566426

FIAP — Análise e Desenvolvimento de Sistemas

Fase: Testes Automatizados

Repositório: github.com/GabrielOlliver/carbon-api-bdd-tests

1. Visão Geral

A Carbon API é uma API REST em .NET 8 para gestão de emissões, reduções e compensações de carbono, com persistência em Oracle 23ai. Nesta entrega foi construída uma suíte de testes automatizados em BDD (Behavior Driven Development): o comportamento esperado da API é descrito em linguagem natural (Gherkin, em português) e cada frase é ligada a código C# que executa chamadas HTTP reais e valida o resultado.

Atendimento aos requisitos do desafio

Requisito do desafio	Como foi atendido
Mínimo de 3 cenários BDD em Gherkin, positivos e negativos	34 cenários (38 execuções, contando as linhas de Exemplos) em 7 funcionalidades, marcados com @positivo e @negativo
Testes de API para todas as APIs	Os 16 endpoints de negócio + 2 endpoints de saúde
Validação de status code	Passo: Então o status da resposta deve ser <código>
Validação do corpo JSON	Passo: E o campo "x" da resposta deve ser "y" (com comparação numérica)
Testes de contrato com JSON Schema	12 schemas draft-07 validados com NJsonSchema em cada resposta
Execução simples e documentada	Um script local (scripts/run-bdd-tests.sh) e um pipeline no GitHub Actions

Ferramentas

Ferramenta	Papel
Reqnroll 3.3.4	Framework BDD para .NET. Sucessor open-source do SpecFlow (sugerido no enunciado, descontinuado em 2024), com a mesma sintaxe Gherkin e os mesmos conceitos (features, steps, hooks)
xUnit	Executor dos testes via dotnet test
HttpClient	Chamadas HTTP reais à API
NJsonSchema 11	Validação de contrato (JSON Schema draft-07)
FluentAssertions	Asserções legíveis, com mensagens que mostram o corpo da resposta em caso de falha
Docker Compose	Sobe a API e o Oracle para os testes
GitHub Actions	Execução automática a cada push

Integração com ESG — pilar Governança e compliance

Os cenários com a tag @governanca garantem integridade, rastreabilidade e conformidade dos dados ambientais: não há empresas com CNPJ duplicado; o histórico de emissões de uma empresa contém apenas registros dela; emissões com quantidade inválida ou data futura são rejeitadas; agendamentos cancelados não podem ser alterados; e os indicadores publicados são matematicamente coerentes (saldo líquido = total emitido − total compensado). Também há ganho de eficiência computacional: a stack de teste é efêmera e os cenários rodam em paralelo.

2. Arquitetura da Suíte de Testes

O fluxo de um cenário típico:

```
Arquivo .feature (Gherkin)     Step Definitions (C#)              Stack sob teste
-------------------------      ----------------------            ----------------
Dado que estou autenticado --> AutenticacaoSteps --POST /login--> carbon-api :8080
E que existe uma empresa   --> PreCondicoesSteps --POST ------->        |
Quando eu envio ... POST   --> RequisicaoSteps   --HTTP ------->        v
Então o status ... 201     --> ValidacaoSteps (status / JSON)      oracle-db
E ... contrato "x.json"    --> ValidacaoSteps --> NJsonSchema --> Schemas/x.json

ApiContext (1 instância por cenário): token JWT, última resposta, variáveis
```

Pasta / arquivo	Responsabilidade
Features/*.feature	Cenários em Gherkin — o QUE a API deve fazer
StepDefinitions/AutenticacaoSteps.cs	Login, ausência de token, token inválido
StepDefinitions/PreCondicoesSteps.cs	Cria empresa, emissão e agendamento pela própria API (passos Dado)
StepDefinitions/RequisicaoSteps.cs	GET / POST / PUT / DELETE genéricos (passos Quando)
StepDefinitions/ValidacaoSteps.cs	Status, campos JSON, listas e contrato (passos Então)
StepDefinitions/GovernancaEsgSteps.cs	Regras de coerência dos indicadores ambientais
Support/ApiContext.cs	Estado do cenário, HttpClient compartilhado, variáveis e leitura de JSON
Support/Hooks.cs	[BeforeTestRun]: verifica /health/ready antes da suíte
Schemas/*.schema.json	Contratos JSON Schema das respostas

Decisões de projeto

- Sem mocks: os testes chamam a API real por HTTP, com o Oracle rodando em container. Isso valida a comunicação entre serviços (API ↔ banco) e o comportamento real das regras de negócio.
- Cenários independentes: cada cenário cria os próprios dados pelos passos Dado. Nenhum cenário depende da ordem de execução, o que permite rodar em paralelo.
- Dados únicos por execução: o CNPJ é gerado aleatoriamente, então a suíte pode ser repetida sem conflito de duplicidade.
- Datas dinâmicas: {ontem}, {amanha}, {daqui90dias} são calculadas na execução — os testes não “envelhecem”.
- Steps reutilizáveis: as frases genéricas permitem criar novos cenários apenas escrevendo Gherkin, sem código C# novo.
- Tags: @positivo, @negativo, @contrato, @governanca e uma tag por funcionalidade permitem executar subconjuntos (dotnet test --filter "Category=governanca").

3. Cenários de Teste em Gherkin

A tabela resume a cobertura das 7 funcionalidades (34 cenários / 38 execuções):

Funcionalidade	Endpoints	Positivos	Negativos
Autenticação	POST /auth/login + proteção JWT	Login válido	Senha incorreta / usuário inexistente (422), sem token (401), token inválido (401)
Empresas	GET lista, GET /{id}, POST	Cadastrar, listar, buscar por id, empresa do seed	CNPJ duplicado (409), id inexistente (404)
Emissões	POST /relatorio-emissao, GET /emissoes-carbono, GET /historico-emissoes/{id}	Registrar, listar, histórico rastreável	Qtd 0/negativa, data futura (422), empresa/histórico inexistente (404)

Agendamentos	POST, GET, GET /{id}, PUT /{id}, DELETE /{id}	Criar, listar, buscar, atualizar, cancelar	Fim antes do início, meta 0 (422), alterar cancelado, cancelar 2×, inexistente
Compensações	POST, GET	Registrar reflorestamento, listar	Quantidade 0 (422), empresa inexistente (404)
Indicadores	GET /indicadores- sustentabilidade, GET /impacto-mitigacao/{id}	Indicadores coerentes, cálculo do impacto	Agendamento inexistente (404)
Saúde	GET /health, GET /health/ready	API no ar, API conectada ao banco	—

3.1 Autenticação — 01_Autenticacao.feature (trecho)

```
# language: pt
@autenticacao @governanca
Funcionalidade: Autenticação de usuários
  Como gestor de sustentabilidade de uma empresa
  Quero me autenticar na Carbon API
  Para que somente pessoas autorizadas registrem e consultem dados ambientais

  @positivo @contrato
  Cenário: Login com credenciais válidas retorna um token JWT
    Quando eu faço login com o email "admin@esg.com" e a senha "admin123"
    Então o status da resposta deve ser 200
    E a resposta deve respeitar o contrato "login.schema.json"
    E o campo "tipo" da resposta deve ser "Bearer"
    E o campo "role" da resposta deve ser "ADMIN"
    E o campo "token" da resposta deve estar preenchido

  @negativo @contrato
  Esquema do Cenário: Login com credenciais inválidas é recusado
    Quando eu faço login com o email "<email>" e a senha "<senha>"
    Então o status da resposta deve ser 422
    E a resposta deve respeitar o contrato "erro.schema.json"
    E a mensagem de erro deve conter "Email ou senha inválidos"
    Exemplos:
      | situacao            | email             | senha       |
      | senha incorreta     | admin@esg.com     | senhaErrada |
      | usuário inexistente | ninguem@esg.com   | admin123    |
```

3.2 Empresas — 02_Empresas.feature (trecho)

```
  @positivo @contrato
  Cenário: Cadastrar uma nova empresa
    Quando eu envio uma requisição POST para "/api/v1/empresas-sustentaveis" com o corpo:
      """
      { "nome": "Solar Verde Ltda", "cnpj": "{cnpjUnico}",
        "setor": "Energia Solar", "pais": "Brasil" }
      """
    Então o status da resposta deve ser 201
    E a resposta deve respeitar o contrato "empresa.schema.json"
    E o campo "nome" da resposta deve ser "Solar Verde Ltda"
    E o campo "ativa" da resposta deve ser "true"

  @negativo @contrato @governanca
  Cenário: Não permitir duas empresas com o mesmo CNPJ
    Dado que existe uma empresa cadastrada
    Quando eu envio uma requisição POST ... com o mesmo CNPJ
    Então o status da resposta deve ser 409
    E a mensagem de erro deve conter "já cadastrada"
```

3.3 Emissões — 03_EmissoesCarbono.feature (trecho)

```
  @positivo @contrato @governanca
  Cenário: Histórico de emissões contém somente registros da empresa consultada
    Dado que existe uma emissão registrada para essa empresa
    Quando eu envio uma requisição GET para "/api/v1/historico-emissoes/{empresaId}"
    Então o status da resposta deve ser 200
    E todos os itens da lista "data" devem ter o campo "empresaId"
      igual a "{empresaId}"

  @negativo @contrato @governanca
  Esquema do Cenário: Rejeitar registro de emissão inválido
    Exemplos: quantidade zero | negativa | data no futuro  →  422
```

Demais funcionalidades (Agendamentos, Compensações, Indicadores e Saúde) seguem o mesmo padrão: cenários positivos com validação de contrato e cenários negativos cobrindo regras de negócio (422), recursos inexistentes (404) e restrições de governança.

4. Testes de Contrato (JSON Schema)

Cada resposta de sucesso ou erro é validada contra um schema draft-07. Exemplo do schema de empresa:

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["id", "nome", "cnpj", "setor", "pais", "ativa", "dataCadastro"],
  "properties": {
    "id":   { "type": "integer", "minimum": 1 },
    "nome": { "type": "string", "minLength": 1 },
    "cnpj": { "type": "string", "pattern": "^\\d{2}\\.\\d{3}\\.\\d{3}/\\d{4}-\\d{2}$" },
    "ativa": { "type": "boolean" },
    "dataCadastro": { "type": "string", "pattern": "^\\d{4}-\\d{2}-\\d{2}T..." }
  }
}
```

Implementação do passo (ValidacaoSteps.cs):

```
[Then("a resposta deve respeitar o contrato {string}")]
public async Task EntaoARespostaDeveRespeitarOContrato(string arquivo)
{
    var caminho = Path.Combine(AppContext.BaseDirectory, "Schemas", arquivo);
    var schema = await JsonSchema.FromFileAsync(caminho);
    var erros = schema.Validate(_api.ResponseBody);
    erros.Should().BeEmpty("a resposta deve seguir o contrato {0}...", arquivo);
}
```

5. Configurações e Execução

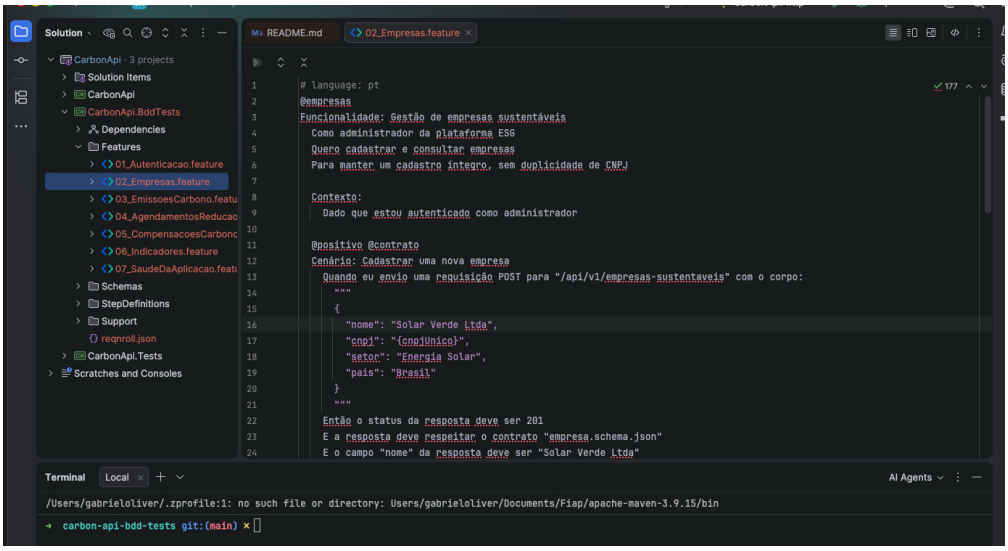


Figura 1 — Projeto CarbonApi.BddTests na solução e arquivo .feature no Rider.

reqnroll.json — Gherkin em português e relatório HTML

```
{
  "$schema": "https://schemas.reqnroll.net/reqnroll-config-latest.json",
  "language": { "feature": "pt" },
  "formatters": {
    "html": { "outputFilePath": "reqnroll_report.html" }
  }
}
```

Pacotes principais (CarbonApi.BddTests.csproj)

```
<PackageReference Include="Reqnroll.xUnit" Version="3.3.4" />
<PackageReference Include="FluentAssertions" Version="6.12.0" />
<PackageReference Include="NJsonSchema" Version="11.0.2" />
<PackageReference Include="xunit" Version="2.9.2" />
```

Script local — scripts/run-bdd-tests.sh

O script sobe a stack (API + Oracle) com Docker Compose, aguarda /health/ready e executa os cenários BDD. Ao final, publica o relatório HTML do Reqnroll.

Pipeline GitHub Actions (trecho do job BDD)

```
bdd-tests:
  name: Testes BDD de API (Gherkin + JSON Schema)
  runs-on: ubuntu-latest
  needs: build-test
  steps:
    - uses: actions/checkout@v4
    - name: Gerar .env efêmero (senhas aleatórias)
      run: |
        cat > .env <<ENV
        ORACLE_PASSWORD=Ora$(openssl rand -hex 8)
        APP_USER_PASSWORD=App$(openssl rand -hex 8)
        JWT_SECRET=$(openssl rand -hex 32)
        ENV
    - run: docker compose up -d --build
    - name: Aguardar /health/ready
      run: |  # loop de até 10 min
    - uses: actions/setup-dotnet@v4
    - run: dotnet test tests/CarbonApi.BddTests -c Release
    - uses: actions/upload-artifact@v4  # TRX + HTML
    - if: always()
      run: docker compose down -v
```

6. Evidências de Execução

Resultado: 38/38 cenários aprovados (100%), tanto localmente quanto no CI.

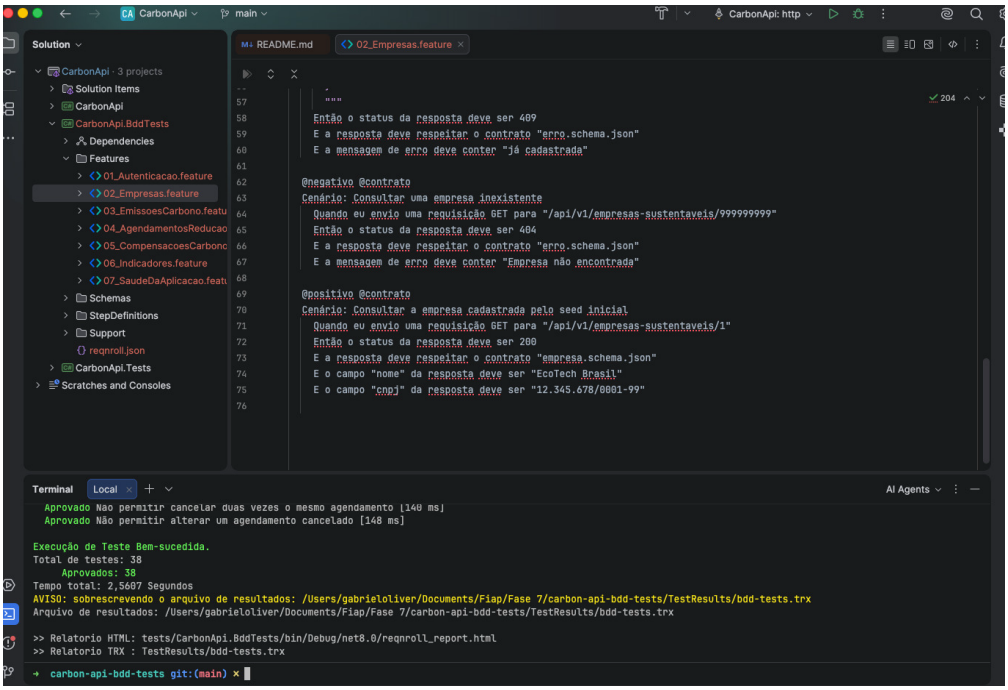


Figura 2 — Execução local pelo script: 38 testes aprovados.

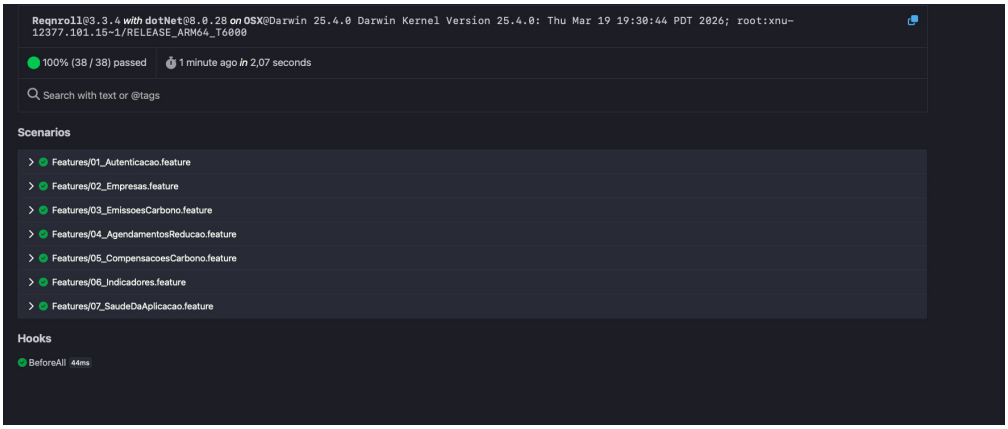


Figura 3 — Relatório HTML gerado pelo Reqnroll: 100% (38/38) aprovados nas 7 funcionalidades.

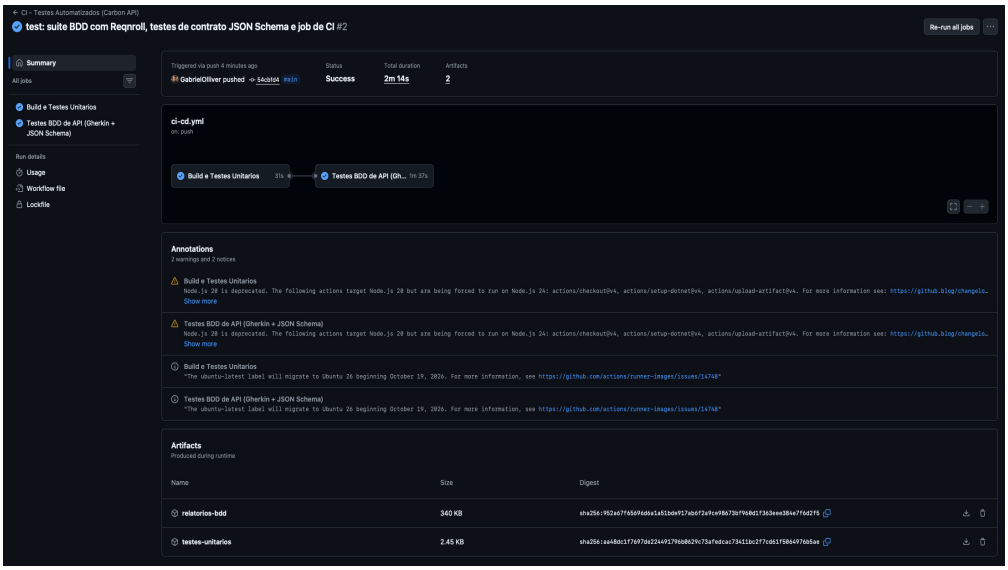


Figura 4 — Pipeline: Build e Testes Unitários → Testes BDD de API, ambos com sucesso.

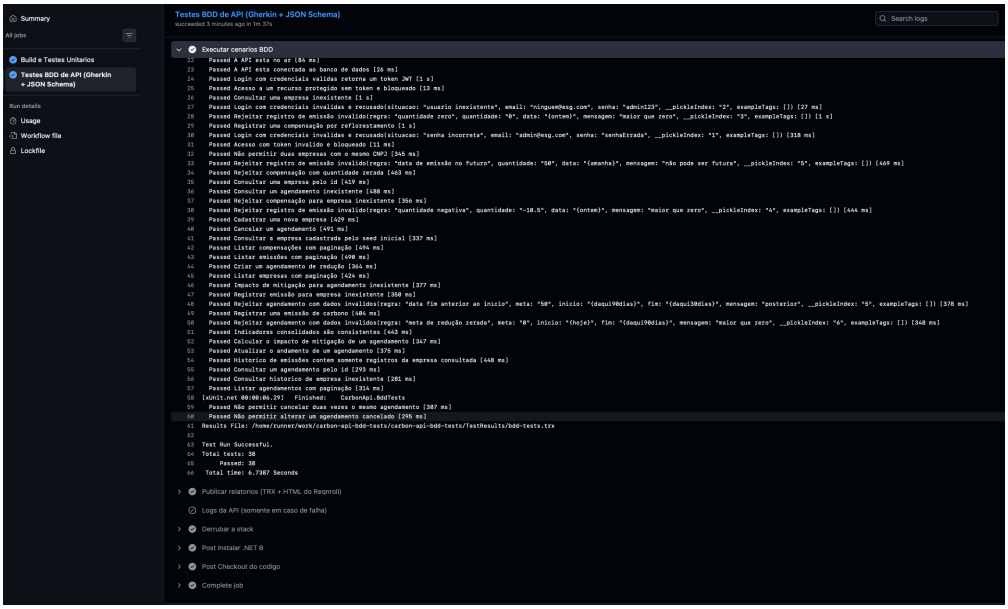


Figura 5 — Log do job BDD no runner: cenários aprovados, total 38/38.



Figura 6 — Relatórios publicados como artifacts (relatorios-bdd e testes-unitarios).

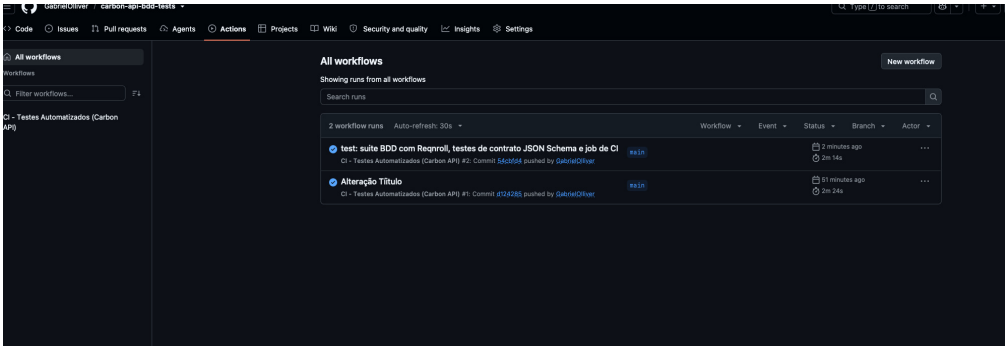


Figura 7 — Histórico de execuções do workflow no repositório.

7. Desafios e Soluções

Desafio	Solução
SpecFlow, sugerido no enunciado, foi descontinuado	Adoção do Reqnroll, seu sucessor open-source compatível (mesma sintaxe Gherkin e mesmos conceitos).
Definir o resultado esperado de cada caso negativo	Leitura do ExceptionMiddleware e dos services: regra de negócio = 422, não encontrado = 404, duplicidade = 409, sem token = 401.
Testes quebrando ao reexecutar (CNPJ duplicado)	Geração de CNPJ aleatório por cenário ({cnpjUnico}).
Dependência entre requisições (PUT/DELETE exigem um id)	ApiContext por cenário guarda {empresaId} e {agendamentoId} criados nos passos Dado.
Oracle devolve decimais com escala fixa (150.5 → 150.5000)	Comparação numérica quando o campo JSON é número, em vez de textual.
Banco precisa estar pronto antes dos testes	Hook [BeforeTestRun] aguarda /health/ready; no CI um passo dedicado espera a stack antes de executar.
Segredos no CI	O pipeline gera um .env efêmero com senhas aleatórias a cada execução; nada sensível fica no repositório.

8. Como Executar

```
git clone https://github.com/GabrielOlliver/carbon-api-bdd-tests.git
cd carbon-api-bdd-tests
cp .env.example .env              # ajustar senhas e JWT_SECRET
./scripts/run-bdd-tests.sh        # sobe API + Oracle, aguarda e executa

# por tag
dotnet test tests/CarbonApi.BddTests --filter "Category=governanca"

# contra outro ambiente
API_BASE_URL=http://servidor:8081 dotnet test tests/CarbonApi.BddTests
```

Anexo A — Catálogo completo dos cenários BDD

Lista de **todos os 34 cenários** das 7 funcionalidades (38 execuções, contando cada linha de *Exemplos* dos Esquemas do Cenário). Os arquivos .feature completos estão em **tests/CarbonApi.BddTests/Features/** no repositório. Legenda das validações: **S** = status code, **J** = corpo JSON, **C** = contrato JSON Schema, **ESG** = regra de governança/coerência ambiental.

A.1 Autenticação de usuários — 1 positivo(s), 3 negativo(s)

Como gestor de sustentabilidade de uma empresa, quero me autenticar na Carbon API, para que somente pessoas autorizadas registrem e consultem dados ambientais.

Nº	Cenário	Tipo	Status	Validações
1	**Login com credenciais válidas retorna um token JWT**	Positivo @governanca	200	S · J · C login.schema.json
2	**Login com credenciais inválidas é recusado** Exemplos (2): senha incorreta; usuário inexistente	Negativo (Esquema) @governanca	422	S · J · C erro.schema.json
3	**Acesso a um recurso protegido sem token é bloqueado** Pré-condições: não estou autenticado	Negativo @governanca	401	S
4	**Acesso com token inválido é bloqueado** Pré-condições: estou autenticado com um token inválido	Negativo @governanca	401	S

A.2 Gestão de empresas sustentáveis — 4 positivo(s), 2 negativo(s)

Como administrador da plataforma ESG, quero cadastrar e consultar empresas, para manter um cadastro íntegro, sem duplicidade de CNPJ.

Nº	Cenário	Tipo	Status	Validações
5	**Cadastrar uma nova empresa**	Positivo	201	S · J · C empresa.schema.json
6	**Listar empresas com paginação**	Positivo	200	S · J · C empresa-lista.schema.json
7	**Consultar uma empresa pelo id** Pré-condições: existe uma empresa cadastrada	Positivo	200	S · J · C empresa.schema.json
8	**Não permitir duas empresas com o mesmo CNPJ** Pré-condições: existe uma empresa cadastrada	Negativo @governanca	409	S · J · C erro.schema.json
9	**Consultar uma empresa inexistente**	Negativo	404	S · J · C erro.schema.json
10	**Consultar a empresa cadastrada pelo seed inicial**	Positivo	200	S · J · C empresa.schema.json

A.3 Registro e rastreabilidade de emissões de carbono — 3 positivo(s), 3 negativo(s)

Como responsável pelo inventário de GEE da empresa, quero registrar e consultar as emissões de carbono, para manter um histórico rastreável e confiável para auditorias ambientais.

Nº	Cenário	Tipo	Status	Validações
11	**Registrar uma emissão de carbono**	Positivo	201	S · J · C emissao.schema.json
12	**Listar emissões com paginação** Pré-condições: existe uma emissão registrada para essa empresa	Positivo	200	S · J · C emissao-lista.schema.json
13	**Histórico de emissões contém somente registros da empresa consultada** Pré-condições: existe uma emissão registrada para essa empresa	Positivo @governanca	200	S · J · C emissao-lista.schema.json
14	**Rejeitar registro de emissão inválido** Exemplos (3): quantidade zero; quantidade negativa; data de emissão no futuro	Negativo (Esquema) @governanca	422	S · J · C erro.schema.json
15	**Registrar emissão para empresa inexistente**	Negativo	404	S · J · C erro.schema.json
16	**Consultar histórico de empresa inexistente**	Negativo	404	S · C erro.schema.json

A.4 Agendamentos de redução de emissões — 5 positivo(s), 4 negativo(s)

Como gestor ambiental, quero planejar, acompanhar e cancelar iniciativas de redução de carbono, para cumprir as metas de descarbonização da empresa.

Nº	Cenário	Tipo	Status	Validações
17	**Criar um agendamento de redução**	Positivo	201	S · J · C agendamento.schema.json
18	**Listar agendamentos com paginação** Pré-condições: existe um agendamento de redução cadastrado para essa empresa	Positivo	200	S · J · C agendamento-lista.schema.json
19	**Consultar um agendamento pelo id** Pré-condições: existe um agendamento de redução cadastrado para essa empresa	Positivo	200	S · J · C agendamento.schema.json
20	**Atualizar o andamento de um agendamento** Pré-condições: existe um agendamento de redução cadastrado para essa empresa	Positivo	200	S · J · C agendamento.schema.json
21	**Cancelar um agendamento** Pré-condições: existe um agendamento de redução cadastrado para essa empresa	Positivo	204	S · J
22	**Rejeitar agendamento com dados inválidos** Exemplos (2): data fim anterior ao início; meta de redução zerada	Negativo (Esquema)	422	S · J · C erro.schema.json
23	**Não permitir alterar um agendamento cancelado** Pré-condições: existe um agendamento de redução cadastrado para essa empresa; esse agendamento foi cancelado	Negativo @governanca	422	S · J · C erro.schema.json
24	**Não permitir cancelar duas vezes o mesmo agendamento** Pré-condições: existe um agendamento de redução cadastrado para essa empresa; esse agendamento foi cancelado	Negativo	422	S · J
25	**Consultar um agendamento inexistente**	Negativo	404	S · C erro.schema.json

A.5 Compensação de carbono — 2 positivo(s), 2 negativo(s)

Como gestor ambiental, quero registrar ações de compensação, como reflorestamento e créditos de carbono, para abater as emissões que não puderam ser evitadas.

Nº	Cenário	Tipo	Status	Validações
26	**Registrar uma compensação por reflorestamento**	Positivo	201	S · J · C compensacao.schema.json
27	**Listar compensações com paginação**	Positivo	200	S · C compensacao-lista.schema.json
28	**Rejeitar compensação com quantidade zerada**	Negativo	422	S · J · C erro.schema.json
29	**Rejeitar compensação para empresa inexistente**	Negativo	404	S · C erro.schema.json

A.6 Indicadores de sustentabilidade — 2 positivo(s), 1 negativo(s)

Como auditor ESG, quero consultar indicadores consolidados e o impacto das ações de redução, para verificar se os números publicados são íntegros e coerentes.

Nº	Cenário	Tipo	Status	Validações
30	**Indicadores consolidados são consistentes**	Positivo @governanca	200	S · J · C · ESG indicadores.schema.json
31	**Calcular o impacto de mitigação de um agendamento** Pré-condições: existe uma empresa cadastrada; existe uma emissão registrada para essa empresa; existe um agendamento de redução cadastrado para essa empresa	Positivo @governanca	200	S · J · C · ESG impacto-mitigacao.schema.json
32	**Impacto de mitigação para agendamento inexistente**	Negativo @governanca	404	S · C erro.schema.json

A.7 Saúde da aplicação — 2 positivo(s), 0 negativo(s)

Como equipe de operações, quero verificar se a API e o banco de dados estão disponíveis, para detectar falhas antes que afetem os usuários.

Nº	Cenário	Tipo	Status	Validações
33	**A API está no ar**	Positivo	200	S · J
34	**A API está conectada ao banco de dados**	Positivo	200	S · J